

PROGRAMAÇÃO FRONT END — REACT

Tutorial: Construindo um Jogo da Memória com React

Desafio da aula — clonar um web game com React, seguindo o mesmo método do tutorial oficial do Jogo da Velha

Tutorial original utilizado como referência:
react.dev/learn/tutorial-tic-tac-toe

Tutorial elaborado com apoio de IA (Claude, Anthropic), conforme proposto no desafio da aula.

1. Objetivo deste tutorial

Este material tem como objetivo reproduzir, passo a passo, a mesma metodologia utilizada no tutorial oficial do Jogo da Velha em React (referência da aula), mas aplicada à construção de um jogo diferente: um Jogo da Memória. A ideia é exercitar os mesmos conceitos fundamentais do React — componentes, props, state, eventos e renderização condicional — só que em um contexto novo, o que ajuda a consolidar o entendimento de quando e por que cada conceito é usado.

- Criar e aninhar componentes (Card, Board e App);
- Renderizar listas de componentes a partir de um array, usando `.map()`;
- Passar dados entre componentes através de props;
- Guardar e atualizar informação com `useState`;
- Responder a eventos de clique;
- Fazer renderização condicional (mostrar o símbolo da carta ou não);
- Lidar com lógica assíncrona simples usando `useEffect` (o efeito de "desvirar" cartas erradas).

2. O jogo escolhido: Jogo da Memória

O Jogo da Memória foi escolhido por ser, assim como o Jogo da Velha, simples nas regras mas rico o suficiente para exigir gerenciamento de estado real:

- O tabuleiro tem um conjunto de cartas viradas para baixo, cada uma escondendo um símbolo;
- Cada par de símbolos aparece em exatamente duas cartas;
- O jogador clica em duas cartas por vez: se os símbolos forem iguais, elas permanecem viradas; se forem diferentes, elas voltam a ficar viradas para baixo após um pequeno intervalo;
- O jogo termina quando todos os pares forem encontrados.

Observação Diferente do Jogo da Velha, aqui a lógica de comparação acontece com um pequeno atraso (o jogador precisa de tempo para ver as duas cartas antes delas desvirarem). Isso é uma ótima desculpa para introduzir o `useEffect`, que não aparece no tutorial original.

3. Preparando o ambiente

Seguindo o mesmo espírito usado em aula no CodePen, vamos montar o projeto inteiro em um único arquivo HTML, sem precisar instalar Node.js, npm ou qualquer bundler. Para isso usamos:

- React e ReactDOM importados diretamente via `esm.sh`, como módulos nativos do navegador (a mesma técnica do slide "Pegando na sua caneta de código");
- O Babel Standalone para transformar a sintaxe JSX em tempo real, no próprio navegador — é o equivalente a trocar o preprocessor do CodePen para "Babel".

Esqueleto inicial do arquivo `index.html`

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8" />
  <title>Jogo da Memoria - React</title>
```

```

    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <style>/* estilos do tabuleiro e das cartas */</style>
  </head>
  <body>
    <div id="root"></div>

    <script type="text/babel" data-presets="react" data-type="module">
      import React from "https://esm.sh/react@18.3.1";
      import { createRoot } from "https://esm.sh/react-dom@18.3.1/client";
      import { useState, useEffect } from "https://esm.sh/react@18.3.1";

      // todo o nosso código React entra aqui dentro
    </script>
  </body>
</html>

```

Repare no atributo `data-type="module"` do script: ele permite que o Babel Standalone interprete tanto o JSX quanto os imports de módulos ES dentro do mesmo bloco — sem isso, os dois recursos juntos não funcionam.

4. Passo 1 — Criando o componente Card

Tal como o tutorial original começa pelo componente Square, aqui começamos pelo Card: uma função que retorna um botão com um símbolo fixo, só para garantir que a renderização básica está funcionando.

```

function Card() {
  return <button className="card">🍎</button>;
}

```

Ao renderizar esse componente, você deve ver um único botão quadrado com a fruta 🍎 dentro dele.

5. Passo 2 — Montando o tabuleiro (Board)

Assim como no tutorial original, se tentarmos colocar vários componentes Card lado a lado dentro de um Board, é preciso envolvê-los em um único elemento pai (aqui usamos uma `div` com a classe `board` em vez de um `Fragment`, já que vamos estilizar essa `div` como uma grade CSS):

```

function Board() {
  return (
    <div className="board">
      <Card />
      <Card />
      <Card />
      <Card />
    </div>
  );
}

```

Observação No tutorial do Jogo da Velha, o board era montado copiando e colando o Square nove vezes. Aqui vamos evitar repetição usando uma lista de dados (o baralho de cartas) e o método `.map()` — a técnica de "Renderizando listas" mencionada na aula.

6. Passo 3 — Gerando o baralho dinamicamente

Em vez de escrever cada carta manualmente, criamos uma função `createDeck()` que gera os 8 pares de símbolos, embaralha o array (algoritmo de Fisher-Yates) e devolve uma lista de objetos representando cada carta:

```
const SYMBOLS = ["🍎", "🍌", "🍇", "🍉", "🍓", "🍒", "🍋", "🍍"];

function shuffle(array) {
  const copy = [...array];
  for (let i = copy.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [copy[i], copy[j]] = [copy[j], copy[i]];
  }
  return copy;
}

function createDeck() {
  const pairs = [...SYMBOLS, ...SYMBOLS];
  return shuffle(pairs).map((symbol, index) => ({
    id: index,
    symbol,
    isFlipped: false,
    isMatched: false,
  }));
}
```

Agora o Board pode percorrer essa lista com `.map()` para renderizar uma carta para cada item:

```
function Board({ cards }) {
  return (
    <div className="board">
      {cards.map((card) => (
        <Card key={card.id} card={card} />
      ))}
    </div>
  );
}
```

Observação Cada elemento de uma lista no React precisa de uma prop `key` única — usamos `card.id` para isso. É o que permite ao React saber exatamente qual carta foi alterada, sem precisar re-renderizar a grade inteira a cada clique.

7. Passo 4 — Adicionando estado com useState

Da mesma forma que o tutorial original adiciona o estado count com useState, aqui o componente App (equivalente ao Game) guarda todo o estado da partida:

```
function App() {
  const [cards, setCards] = useState(createDeck);
  const [selected, setSelected] = useState([]);
  const [moves, setMoves] = useState(0);
  const [locked, setLocked] = useState(false);

  return <Board cards={cards} />;
}
```

- cards — a lista de 16 cartas e seus status (virada, encontrada);
- selected — os ids das cartas viradas no momento (no máximo duas);
- moves — contador de jogadas;
- locked — trava o tabuleiro enquanto o jogo está "decidindo" se as duas cartas formam um par.

8. Passo 5 — Passando dados através de props

Repetindo o padrão visto na aula (Board → Square), o App passa os dados para o Board, que repassa cada carta individual para o Card correspondente, junto com uma função de callback para o clique:

```
function Card({ card, onCardClick }) {
  return (
    <button className="card" onClick={() => onCardClick(card.id)}>
      {card.symbol}
    </button>
  );
}

function Board({ cards, onCardClick }) {
  return (
    <div className="board">
      {cards.map((card) => (
        <Card key={card.id} card={card} onCardClick={onCardClick} />
      ))}
    </div>
  );
}
```

9. Passo 6 — Respondendo a eventos: o clique na carta

Dentro do App, criamos a função handleCardClick, que decide o que acontece quando uma carta é clicada. Ela ignora o clique se o tabuleiro estiver travado, se já houver duas cartas selecionadas, ou se a carta clicada já estiver virada ou encontrada:

```
function handleClick(id) {
  if (locked || selected.length >= 2) return;

  const card = cards.find((c) => c.id === id);
  if (!card || card.isFlipped || card.isMatched) return;

  setCards((prevCards) =>
    prevCards.map((c) => (c.id === id ? { ...c, isFlipped: true } : c))
  );
  setSelected((prevSelected) => [...prevSelected, id]);
}
```

10. Passo 7 — Renderização condicional

Assim como o tutorial original usa if para decidir o que mostrar na tela, aqui usamos uma expressão condicional dentro do JSX para decidir se a carta mostra o símbolo (frente) ou fica vazia (verso):

```
function Card({ card, onClick }) {
  const className =
    "card" +
    (card.isFlipped ? " flipped" : "") +
    (card.isMatched ? " matched" : "");

  return (
    <button
      className={className}
      onClick={() => onClick(card.id)}
      disabled={card.isFlipped || card.isMatched}
    >
      {card.isFlipped || card.isMatched ? card.symbol : ""}
    </button>
  );
}
```

11. Passo 8 — useEffect: verificando se as cartas formam um par

Esse é o passo que mais diferencia este projeto do Jogo da Velha. Sempre que selected mudar e tiver exatamente duas cartas, um useEffect entra em ação: trava o tabuleiro, soma uma jogada e compara os símbolos. Se forem iguais, marca as cartas como encontradas na hora; se forem diferentes, espera 800ms (tempo para o jogador memorizar) antes de virá-las de volta:

```
useEffect(() => {
  if (selected.length !== 2) return;

  setLocked(true);
  setMoves((prevMoves) => prevMoves + 1);
```

```

const [firstId, secondId] = selected;
const firstCard = cards.find((card) => card.id === firstId);
const secondCard = cards.find((card) => card.id === secondId);

if (firstCard.symbol === secondCard.symbol) {
  setCards((prevCards) =>
    prevCards.map((card) =>
      card.id === firstId || card.id === secondId
        ? { ...card, isMatched: true }
        : card
    )
  );
  setSelected([]);
  setLocked(false);
} else {
  const timer = setTimeout(() => {
    setCards((prevCards) =>
      prevCards.map((card) =>
        card.id === firstId || card.id === secondId
          ? { ...card, isFlipped: false }
          : card
      )
    );
    setSelected([]);
    setLocked(false);
  }, 800);

  return () => clearTimeout(timer);
}
}, [selected]);

```

Observação O retorno da função dentro do `useEffect` (`return () => clearTimeout(timer)`) é a função de limpeza do efeito — ela cancela o timer pendente caso o componente seja desmontado ou o efeito rode de novo antes do tempo acabar, evitando comportamentos inesperados.

12. Passo 9 — Placar e mensagem de vitória

Para fechar a experiência, calculamos quantos pares já foram encontrados e exibimos um contador de jogadas e uma mensagem de vitória quando todos os pares forem descobertos:

```

const totalPairs = SYMBOLS.length;
const matchedCount = cards.filter((card) => card.isMatched).length / 2;
const won = matchedCount === totalPairs;

return (
  <div className="app">
    <h1>Jogo da Memória</h1>

    <div className="status-bar">
      <span>Jogadas: {moves}</span>
      <span>Pares: {matchedCount} / {totalPairs}</span>
    </div>
  </div>
)

```

```

    </div>

    <Board cards={cards} onCardClick={handleCardClick} />

    {won && <p className="win-message">Você venceu em {moves} jogadas! 🎉</p>}
  </div>
);

```

13. Passo 10 — Botão de reiniciar

Por fim, adicionamos um botão que recria o baralho do zero e zera o placar:

```

function handleRestart() {
  setCards(createDeck());
  setSelected([]);
  setMoves(0);
  setLocked(false);
}

// ...
<button className="restart-button" onClick={handleRestart}>
  Reiniciar jogo
</button>

```

14. Código final completo

Juntando todos os passos anteriores, o componente App fica assim (o componente Card e Board permanecem exatamente como mostrado nos passos 5 e 7):

App.jsx (código completo)

```

function App() {
  const [cards, setCards] = useState(createDeck);
  const [selected, setSelected] = useState([]);
  const [moves, setMoves] = useState(0);
  const [locked, setLocked] = useState(false);

  const totalPairs = SYMBOLS.length;
  const matchedCount = cards.filter((card) => card.isMatched).length / 2;
  const won = matchedCount === totalPairs;

  function handleCardClick(id) {
    if (locked || selected.length >= 2) return;

    const card = cards.find((c) => c.id === id);
    if (!card || card.isFlipped || card.isMatched) return;

    setCards((prevCards) =>
      prevCards.map((c) => (c.id === id ? { ...c, isFlipped: true } : c))
    );
  }
}

```



```

    );
    setSelected((prevSelected) => [...prevSelected, id]);
  }

  useEffect(() => {
    if (selected.length !== 2) return;

    setLocked(true);
    setMoves((prevMoves) => prevMoves + 1);

    const [firstId, secondId] = selected;
    const firstCard = cards.find((card) => card.id === firstId);
    const secondCard = cards.find((card) => card.id === secondId);

    if (firstCard.symbol === secondCard.symbol) {
      setCards((prevCards) =>
        prevCards.map((card) =>
          card.id === firstId || card.id === secondId
            ? { ...card, isMatched: true }
            : card
        )
      );
      setSelected([]);
      setLocked(false);
    } else {
      const timer = setTimeout(() => {
        setCards((prevCards) =>
          prevCards.map((card) =>
            card.id === firstId || card.id === secondId
              ? { ...card, isFlipped: false }
              : card
          )
        );
      });
      setSelected([]);
      setLocked(false);
    }, 800);

    return () => clearTimeout(timer);
  }, [selected]);

  function handleRestart() {
    setCards(createDeck());
    setSelected([]);
    setMoves(0);
    setLocked(false);
  }

  return (
    <div className="app">
      <h1>Jogo da Memória</h1>

      <div className="status-bar">
        <span>Jogadas: {moves}</span>
        <span>Pares: {matchedCount} / {totalPairs}</span>
      </div>
    </div>
  );

```

```

    </div>

    <Board cards={cards} onCardClick={handleCardClick} />

    {won && <p className="win-message">Você venceu em {moves} jogadas! 🎉</p>}

    <button className="restart-button" onClick={handleRestart}>
      Reiniciar jogo
    </button>
  </div>
);
}

const root = createRoot(document.getElementById("root"));
root.render(<App />);

```

O arquivo completo — incluindo o HTML, o CSS do tabuleiro e o JavaScript inteiro — está disponível no projeto implementado, no arquivo `index.html`.

15. Testando o projeto

- Abra o arquivo `index.html` diretamente no navegador (duplo clique) — não é necessário instalar nada;
- Clique em duas cartas por vez para tentar formar pares;
- Acompanhe o contador de jogadas e de pares encontrados no topo do tabuleiro;
- Use o botão Reiniciar jogo para embaralhar as cartas novamente a qualquer momento.

Observação O projeto foi testado de ponta a ponta de forma automatizada (simulando cliques em React) antes da entrega, cobrindo: renderização do baralho, acerto de par, erro de par com desvirada automática, bloqueio de cliques durante a verificação, contagem de jogadas, condição de vitória e reinício do jogo.

16. Subindo o tutorial e o projeto no Git

Conforme pedido no desafio da aula, depois de validar o projeto localmente:

```

git init
git add .
git commit -m "Jogo da Memória com React - desafio da aula"
git branch -M main
git remote add origin https://github.com/SEU-USUARIO/jogo-da-memoria-react.git
git push -u origin main

```

- Inclua tanto a pasta do projeto (`index.html`, `README.md`) quanto este tutorial em PDF no repositório;
- Um bom nome de repositório: `jogo-da-memoria-react`.

17. Conclusão e próximos passos

Com esse exercício, os mesmos conceitos do tutorial do Jogo da Velha — componentes, props, state, eventos e renderização condicional — foram aplicados em um contexto diferente, e ainda foi necessário aprender uma ferramenta nova (useEffect) para lidar com o comportamento assíncrono do jogo. Algumas ideias para evoluir o projeto:

- Adicionar um cronômetro para medir o tempo de cada partida;
- Criar níveis de dificuldade (mais ou menos pares de cartas);
- Guardar o melhor resultado (recorde) usando localStorage;
- Adicionar efeitos sonoros ao acertar ou errar um par;
- Criar um modo para dois jogadores, alternando a vez a cada erro.